Logistic Regression

Logistic Regression

- Despite its name, logistic regression is a classification algorithm
- Simple and effective algorithm for binary classification
- Can be adapted to multiclass problems
- Works with real-valued attributes only
- One of the simplest examples of learning algorithms based on **gradient descent**
- Also known as single-layer perceptron
- Starting point for neural networks and deep learning

Logistic Regression for Binary Classification

We use the logistic function to estimate the probability that example *x* belongs to class 1 using the logistic or sigmoid function:

$$p(c(x) == 1) = \sigma(x \cdot w) = \frac{1}{1 + e^{-x \cdot w}}$$

Since classes are binary, we apply a threshold to the probability to make a prediction:

$$pred(x) = \begin{cases} 1 & \text{if } p(c(x) == 1) > \frac{1}{2} \\ 0 & \text{otherwise} \end{cases}$$

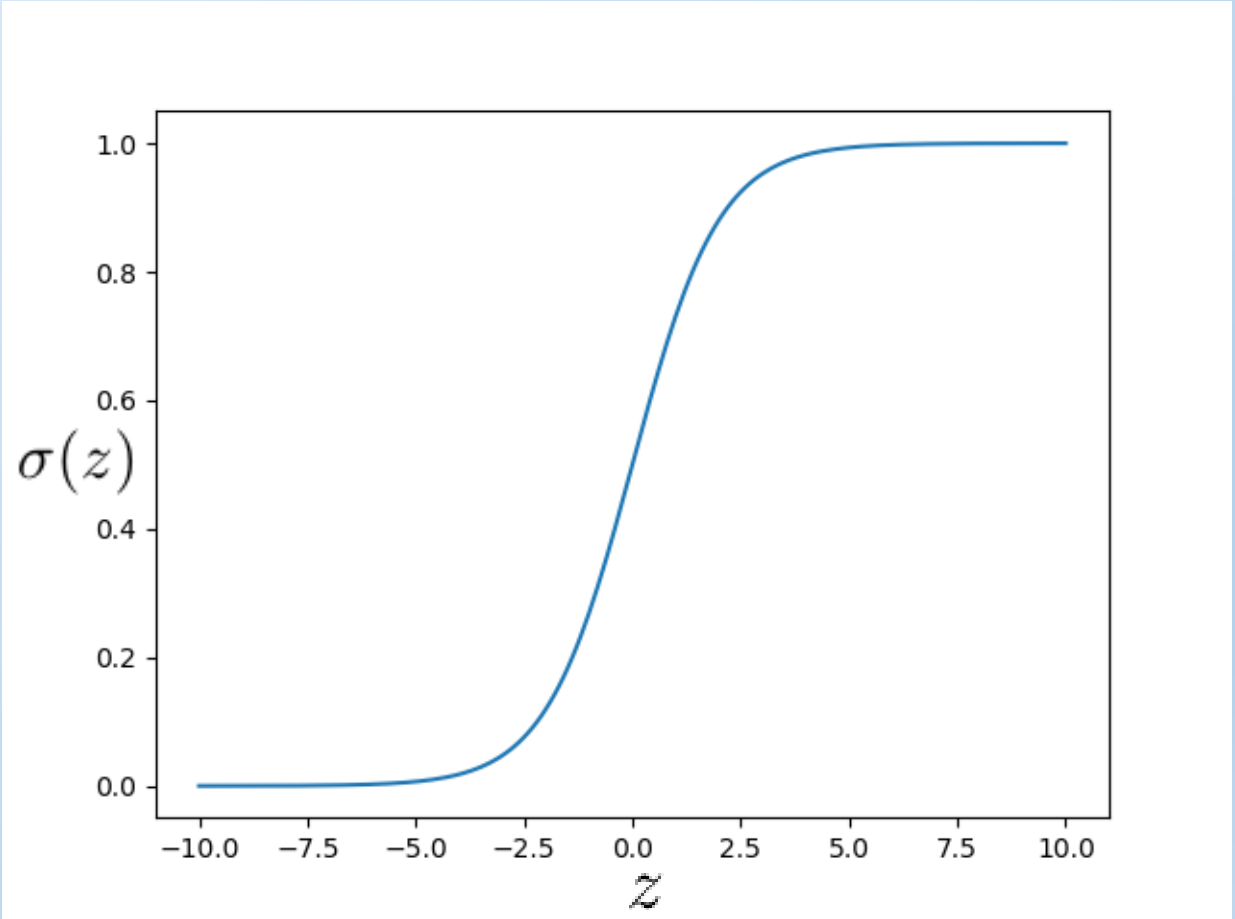
Where $\boldsymbol{x \cdot w}$ is the dot product:

$\boldsymbol{x \cdot w} = x_0 \cdot w_0 + x_1 \cdot w_1 + \ldots + x_{n-2} \cdot w_{n-2} + x_{n-1} \cdot w_{n-1}$

and the parameters $w_0, w_1, \ldots, w_{n-2}, w_{n-1}$ are learned from the training data

The Logistic Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Logistic Regression Learning

We define the error function as the mean squared error, as in linear regression – but consider the output before applying the threshold, since the actual prediction is not differentiable:

$$E(X, y, w) = \frac{1}{2n} \sum_{i=0}^{n-1} (\sigma(x_i \cdot w) - y_i)^2 \qquad \longrightarrow E(X, y, w) \text{ is the mean squared error}$$

Let's define a new variable $z_i = x_i \cdot w$

$$E(X, y, w) = \frac{1}{2n} \sum_{i=0}^{n-1} (\sigma(z_i) - y_i)^2$$

$$\frac{d(E,y,w)}{dw} = \frac{1}{2n} \sum_{i=0}^{n-1} 2\,(\sigma(z_i) - y_i) \frac{d\sigma(z_i)}{dz_i} \frac{dz_i}{dw}$$

$$\frac{d(E,y,w)}{dw} = \frac{1}{n} \sum_{i=0}^{n-1} (\sigma(z_i) - y_i) \frac{d\sigma(z_i)}{dz_i} \frac{dz_i}{dw}$$

$$\frac{d\sigma(z_i)}{dz_i} = \sigma(zi)(1 - \sigma(z_i))$$

$$\frac{dz_i}{dw} = [x_{i0}, xi_{1,\dots}, x_{im_1}]$$

Logistic Regression

Finally:

$$\frac{d(E,y,w)}{dw} = \frac{1}{n} \sum_{i=0}^{n-1} (\sigma(z_i) - y_i)\, \sigma(z_i)\big(1 - \sigma(z_i)\big)[x_{i0}, xi_1, \cdots, x_{im\ _\ 1}]$$

scalar

vector of partial derivatives of length *m*

Logistic Regression – Python Code

```
def sigma(X,w):
    z = np.sum(X*w, axis =1)
    return (1/(1+np.exp(-z)))


def logistic_regression_gd(X,y,lr=0.01, tol =0.001,max_it = 1000):
    n, m = X.shape          # the training set has n examples and m attributes
    w =  (np.random.random((1,m))-.5)*.01    # w is 1xm
    for i in range(1,max_it+1):
        s = sigma(X,w)          # s has len n
        error = (s - y)         # error is nx1
        if np.mean(error**2)<tol: break
        g = (error*s*(1-s)).reshape(-1,1)
        gradient = np.mean(X*g,axis=0)   # gradient is 1xm
        w = w - lr*gradient
   return w
```

Logistic Regression

Multi-class classification

- Use a one-hot representation of the output
- Algorithms learns an array of size k-by-m, where k is the number of classes and m is the number of features
- The logistic function is replaced by the softmax function:

$$s\left(x_i\right) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

Multiclass Logistic Regression – Python Code

```
def sigma(X,W):
    z = np.matmul(X,W.T)
    return (1/(1+np.exp(-z)))


def logistic_regression_gd(X,y,lr=0.01, tol =0.001,max_it = 1000,display_period=10):
    n, m = X.shape                          # the training set has n examples and m attributes
    y_oh = one_hot(y)                       # y_oh has n rows and k=num_classes columns
    W =  (np.random.random((y_oh.shape[1],m))-.5)*.001     # W is k-by-m
    for i in range(1,max_it+1):
        S = sigma(X,W)                     # S is n-by-k
        Error = (S - y_oh)                  #  Error is n-by-k
        if np.mean(Error**2)<tol: break
        G = (Error*S*(1-S)).T               #  G is k-by-n
        Gradient = np.matmul(G,X)/n    # gradient is k-by-m
        W = W - lr*Gradient
    return W
```

Logistic Regression

Some Improvements:

- Regularization:
  - Add a term to the loss function to penalize square weights (L2 regularization), the absolute values of the weights (L1 regularization), or a combination of the two
- Batch training:
  - Randomly split training data into small groups – batches – and update weight matrix after processing each batch – more weight updates, randomness may help avoid local minima
- Adaptive learning rate:
  - Reduce learning rate after a fixed number of epochs
  - Reduce learning rate when MSE does not decrease after a certain number of epochs
- Momentum:
  - Keep a running estimate of the gradient, combining the previous estimate and the gradient computed from the current batch
- Second order approximation of gradient:
  - Use second derivatives to obtain a good approximation of gradient in a larger region
- Label Smoothing:
  - y_smooth = y_oh * α + $\frac{(1-\alpha)}{\text{n_classes}}$ for $\alpha < 1$ - Notice that $\sigma(z) = 1$ when $z = \infty$